Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра информационных технологий автоматизированных систем

ОТЧЕТ

по практическому занятию № 1
«*Middleware* и шаблонизаторы»
по дисциплине «Интернет-технологии и веб-программирование»

Выполнила — студентка гр. 320604
К.А. Хаджинова

Проверил — А.Л. Гончаревич

Минск 2026

СОДЕРЖАНИЕ

1 Цель работы 3
2 Краткие теоретические сведения 4
3 Ход работы 5
3.1 Структура проекта 5
3.2 Журнал запросов в таблице базы данных 5
3.3 Имитация авторизации 6
3.4 Маршруты и страницы 7
3.5 Обработка ошибок 404 и 500 7
3.6 Проверка полей формы 9
3.7 Страницы, сгенерированные сервером 10
4 Ответы на контрольные вопросы 21
Заключение 22
Список использованных источников 23
Приложение А (обязательное) Настройка шаблонизатора и порядок подключения middleware (к подразделу 3.1) 24
Приложение Б (обязательное) Шаблон главной страницы со списком диаграмм (к подразделу 3.1) 26
Приложение В (обязательное) Middleware записи запросов в журнал (к подразделу 3.2) 27
Приложение Г (обязательное) Создание таблицы журнала и запросы к базе SQLite (к подразделу 3.2) 28
Приложение Д (обязательное) Middleware имитации авторизации (к подразделу 3.3) 30
Приложение Е (обязательное) Разбор тела формы и обработчики маршрутов приложения (к подразделу 3.4) 31
Приложение Ж (обязательное) Шаблон детальной страницы диаграммы (к подразделу 3.4) 34
Приложение И (обязательное) Обработчики ошибок 404 и 500 (к подразделу 3.5) 35
Приложение К (обязательное) Хранилище записей и проверка полей формы (к подразделу 3.6) 37
Приложение Л (обязательное) Файл пределов для полей формы и журнала (к подразделу 3.6) 40

1 ЦЕЛЬ РАБОТЫ

Цель работы – освоить концепцию промежуточных обработчиков *(middleware)* в *Express.js*, разработать собственные *middleware* для журналирования запросов, имитации авторизации и обработки ошибок и изучить шаблонизатор *EJS* для генерации динамических *HTML*-страниц на сервере. Работа выполнена на примере проекта *DSLKit* – веб-редактора диаграмм как кода. Тематика проекта относится к направлению *SaaS* и совпадает с темой курсового проекта по варианту 26.

Для достижения цели решались следующие задачи:

– создать *Express*-приложение с поддержкой шаблонизатора *EJS*;

– реализовать *middleware*, которое фиксирует метод, адрес и время каждого запроса;

– реализовать *middleware* для имитации авторизации;

– создать страницы списка, детальной записи и формы добавления с передачей данных из обработчиков маршрутов;

– добавить *middleware* обработки ошибок 404 и 500.

По условию записи каталога хранятся в массиве в памяти без базы данных. Номер варианта 26 четный, поэтому журнал запросов по правилу варианта ведется не в файле, а в таблице базы данных.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Middleware в *Express* – это функция с параметрами *req*, *res* и *next*, которая получает объект запроса и объект ответа. Она может сама завершить ответ или передать управление следующему обработчику вызовом *next()*. Функции выполняются в порядке подключения, поэтому именно последовательность вызовов *app.use()* и объявлений маршрутов определяет поведение приложения [1].

Обработчик ошибок отличается от обычного *middleware* сигнатурой из четырех параметров *(err, req, res, next)*. *Express* передает ему управление, когда предыдущий обработчик вызывает *next(error)* или выбрасывает исключение. По этой причине обработчик ошибок подключают после всех маршрутов [2].

EJS (Embedded JavaScript) – шаблонизатор, в котором *HTML*-разметка сочетается со вставками на *JavaScript*. Запись *<%= value %>* выводит значение с экранированием *HTML*, запись *<% code %>* выполняет код без вывода, а запись *<%- include('partials/header') %>* подключает частичный шаблон. Данные передаются в шаблон вторым аргументом метода *res.render()* [3].

Записи каталога хранятся в массиве в памяти процесса и теряются при перезапуске сервера. Журнал запросов хранится в таблице базы *SQLite*, доступ к которой дает встроенный модуль *node:sqlite*, поэтому отдельный пакет для работы с базой не требуется [4].

3 ХОД РАБОТЫ

3.1 Структура проекта

Код работы находится в ветке *pz21* репозитория *https://github.com/KsushaKhadzhinova/DSLKit-Project* в каталоге *part-2/pz-01-middleware-ejs*. Приложение работает на *Express* 5 с шаблонизатором *EJS* 3 и требует *Node.js* версии 22.13 или новее. Для запуска в каталоге проекта выполняются команды *npm install* и *npm start*, после чего сервер принимает запросы на порту 3000. Назначение основных файлов проекта приведено в таблице 1.

Таблица 1 – Структура проекта практического занятия 1

Файл	Назначение
src/server.js	Запуск сервера на порту
src/app.js	Сборка приложения, порядок *middleware* и маршруты
src/middleware/requestLogger.js	Запись каждого запроса в таблицу журнала
src/middleware/fakeAuth.js	Имитация авторизации
src/middleware/errorHandlers.js	Обработчики ошибок 404 и 500
src/data/diagrams.js	Хранилище записей в памяти и проверка полей формы
src/data/requestLog.js	Таблица *request_log* в *SQLite*
src/errors.js	Пользовательские тексты страниц ошибок
limits.json	Пределы длины полей и размера формы
views/	Шаблоны *EJS* и частичные шаблоны
public/	Стили, логотип и переключатель темы

Шаблонизатор подключен в файле *src/app.js* вызовами *app.set('view engine', 'ejs')* и *app.set('views', ...)*. Вместо общего файла *layout.ejs* использованы частичные шаблоны *header.ejs* и *footer.ejs*. Каждая страница подключает их в начале и в конце, поэтому разметка шапки и подвала не дублируется. Настройка шаблонизатора и порядок подключения *middleware* приведены в приложении А, шаблон главной страницы приведен в приложении Б.

3.2 Журнал запросов в таблице базы данных

Middleware журнала *requestLogger* подключено после статических файлов и имитации авторизации. До выполнения маршрута оно вставляет в таблицу *request_log* строку с временем, методом и адресом запроса. По событию *finish* объекта ответа в ту же строку дописываются код статуса и длительность обработки, а итоговая строка выводится в консоль сервера.

Запись выполняется до маршрута намеренно. Если база недоступна, ошибка передается через *next(error)* в обработчик ошибок и пользователь получает оформленную страницу 500. После отправки ответа сообщить о сбое уже невозможно. Если папки для файла базы нет, она создается автоматически, а таблица создается при первом обращении командой *CREATE TABLE IF NOT EXISTS*. Соединение с базой открывается лениво, поэтому недоступная база не мешает запуску сервера.

Значение параметра авторизации в сохраняемом адресе маскируется, а переводы строк и управляющие символы экранируются. Это исключает подделку строк журнала через адрес запроса. Страница */log* показывает последние строки таблицы, их количество задано в файле *limits.json*. *Middleware* журнала приведено в приложении В, модуль работы с таблицей приведен в приложении Г.

3.3 Имитация авторизации

Middleware fakeAuth проверяет *query*-параметр *auth*. При значении 1 в *req.user* записывается владелец *(Owner)*, в остальных случаях записывается гость *(Guest)*. Имя пользователя и окончание адреса *?auth=1* передаются в шаблоны через *res.locals*, поэтому ссылки внутри сайта сохраняют состояние входа.

Маршруты, изменяющие данные, и страница журнала защищены функцией *requireAuth*. Она перенаправляет гостя на страницу */login* с пояснением, как выполнить вход. Код обоих *middleware* приведен в приложении Д.

3.4 Маршруты и страницы

Помимо четырех маршрутов из условия реализованы маршруты изменения и удаления записи и служебные маршруты. Их перечень приведен в таблице 2.

Таблица 2 – Маршруты приложения

Маршрут	Результат
GET /	Список диаграмм из массива с временем создания записей
GET /item/:id	Детальная страница, неизвестный номер ведет на страницу 404
GET /add	Форма добавления, доступна с *?auth=1*
POST /add	Проверка полей, добавление в массив и переход на главную, при ошибке код 400
GET /item/:id/edit	Форма изменения записи с текущими значениями
POST /item/:id/edit	Изменение записи, номер и время создания сохраняются
POST /item/:id/delete	Удаление записи и переход на главную
GET /login	Страница с пояснением входа
GET /log	Таблица журнала запросов, доступна с *?auth=1*

Данные передаются из обработчиков маршрутов в шаблоны вторым аргументом *res.render()*. Маршрут *GET /* передает в шаблон *index.ejs* массив *items*, а маршрут *GET /item/:id* передает найденный объект *item*. Номер записи принимается только в виде последовательности цифр, поэтому адреса вида */item/abc* приводят на страницу 404. Для вывода времени создания в *app.locals* зарегистрирована вспомогательная функция *formatTime*, доступная во всех шаблонах.

На детальной странице авторизованный пользователь видит кнопки «*Edit*» и «*Delete*». Изменение открывает тот же шаблон *add.ejs*, заполненный текущими значениями, и проверяет поля по тем же правилам, что и добавление. Удаление выполнено формой с методом *POST*, а не ссылкой, так как запрос *GET* не должен изменять данные. Обработчики маршрутов приведены в приложении Е, шаблон детальной страницы приведен в приложении Ж.

3.5 Обработка ошибок 404 и 500

Обработчик *notFound* подключен после всех маршрутов и рендерит шаблон *404.ejs* с кодом ответа 404. Обработчик *serverError* имеет четыре параметра и подключен последним, поэтому в него попадает любое

исключение. Это может быть сбой записи в журнал, ошибка внутри маршрута или некорректное тело запроса.

Обработчик выводит в консоль сервера метод, адрес и полный текст системной ошибки, определяет код ответа и рендерит шаблон *500.ejs*. Пользователь видит не системный код вроде *EACCES*, а страницу в общем оформлении сайта с шапкой и подвалом. На странице указаны код ошибки и три смысловых блока: что произошло, почему это могло случиться и как это исправить. Тексты этих блоков собраны в модуле *src/errors.js*. Если не удалось отрисовать даже шаблон 500, отправляется запасная страница с теми же стилями. Поэтому вместо оформленной страницы никогда не появляется голый текст на черном фоне.

Страница 500 вызывается настоящим сбоем базы журнала, а не кнопкой. Системная ошибка файловой системы переводится в пользовательское сообщение функцией *fileFailure*. Соответствие типовых поломок и сообщений, которые видит пользователь, приведено в таблице 3.

Таблица 3 – Реакция сервера на поломку файла базы журнала

Поломка	Системный код	Сообщение для пользователя
Папки базы нет при первом обращении	–	Папка и таблица создаются автоматически, ошибки нет
Файл базы удален, перемещен или заблокирован другой программой	Ошибка *SQLite*	База журнала недоступна, проверьте файл и папку базы
На месте файла базы создана папка с тем же именем	*EISDIR*	На месте файла журнала находится папка
Нет прав на папку или в пути вместо папки стоит файл	*EACCES, EPERM, ENOTDIR, EROFS*	Нет доступа к папке журнала
Диск заполнен или файл занят	*ENOSPC, EBUSY*	База журнала недоступна, проверьте файл и папку базы

Для демонстрации на защите сервер запускается командой *DB_FILE=package.json/blocked/log.db npm start*. Путь проходит через обычный файл *package.json*, поэтому папку для базы создать невозможно и любой запрос завершается страницей 500. Обработчики ошибок приведены в приложении И.

3.6 Проверка полей формы

Форма добавления проверяется на сервере, а те же пределы передаются в разметку атрибутами *maxlength*, *min* и *max*. Перед проверкой значения обрезаются по краям, поэтому поле из одних пробелов считается пустым. Правила проверки приведены в таблице 4.

Таблица 4 – Проверки полей формы добавления

Поле	Правило
Название	Обязательно, непустое после обрезки пробелов, не длиннее 80 символов, без управляющих символов
Описание	Обязательно, непустое после обрезки пробелов, не длиннее 500 символов
Нотация	Одно значение из списка *UML, BPMN, ERD, DFD, IDEF0, Petri*
Код диаграммы	Необязательно, не длиннее 5000 символов
Ревизия	Обязательна, целое число от 0 до 100000, отрицательные и дробные значения отвергаются
Тело формы	Не больше 20 КБ и не больше 20 полей

Ограничение длины и размера тела формы не дает перегрузить сервер вставкой огромного текста или числа. При ошибке сервер отвечает кодом 400, повторно показывает форму с введенными значениями и выводит сообщение под каждым неверным полем. Каждая запись получает поле «время создания», которое всегда позже предыдущего, поэтому одинаковые по содержанию записи различаются визуально. Все пределы хранятся в файле *limits.json*, а не в коде. Код проверки приведен в приложении К, файл пределов приведен в приложении Л.

3.7 Страницы, сгенерированные сервером

Шапка сайта закреплена при прокрутке, а в подвале расположены ссылки. На снимках видны оба элемента, пустой фон между содержимым и подвалом сокращен. Главная страница со списком диаграмм показана на рисунке 1. На ней три исходные записи с нотацией, номером и временем создания и кнопка «*Add diagram*».

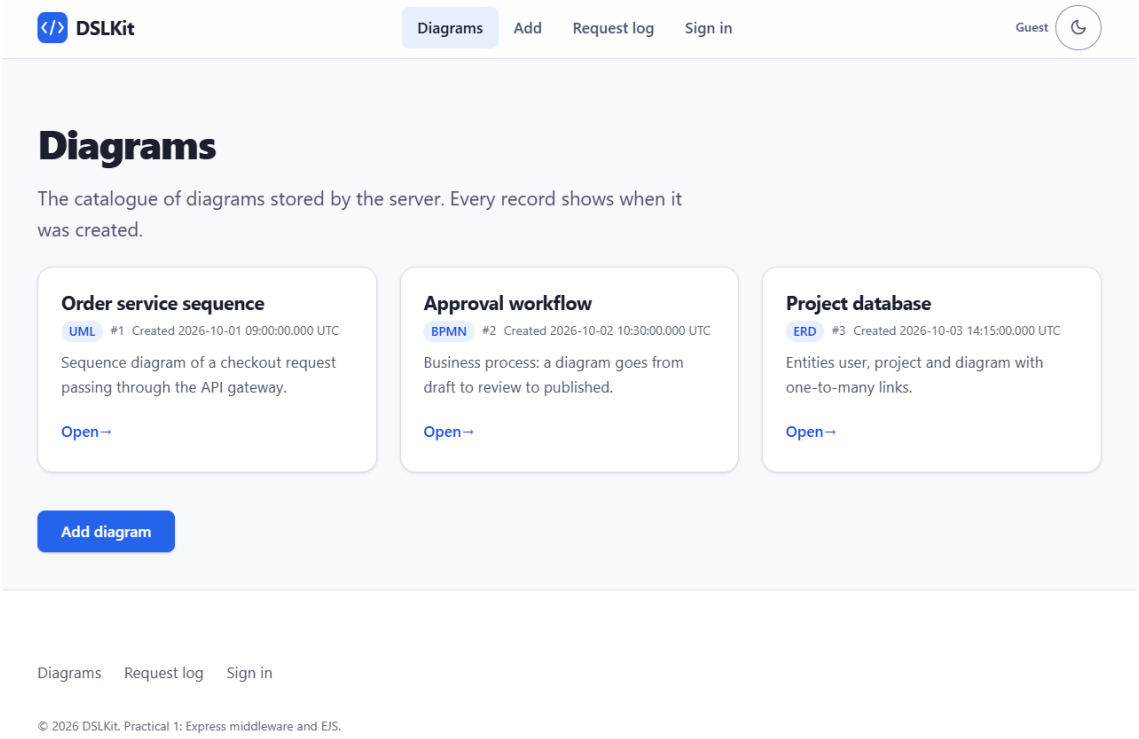


Рисунок 1 – Главная страница со списком из трех диаграмм и временем их создания

Детальная страница записи показана на рисунке 2. В шаблон *item.ejs* подставлены значения объекта *item*, а кнопки «*Edit*» и «*Delete*» скрыты, так как страница открыта гостем.

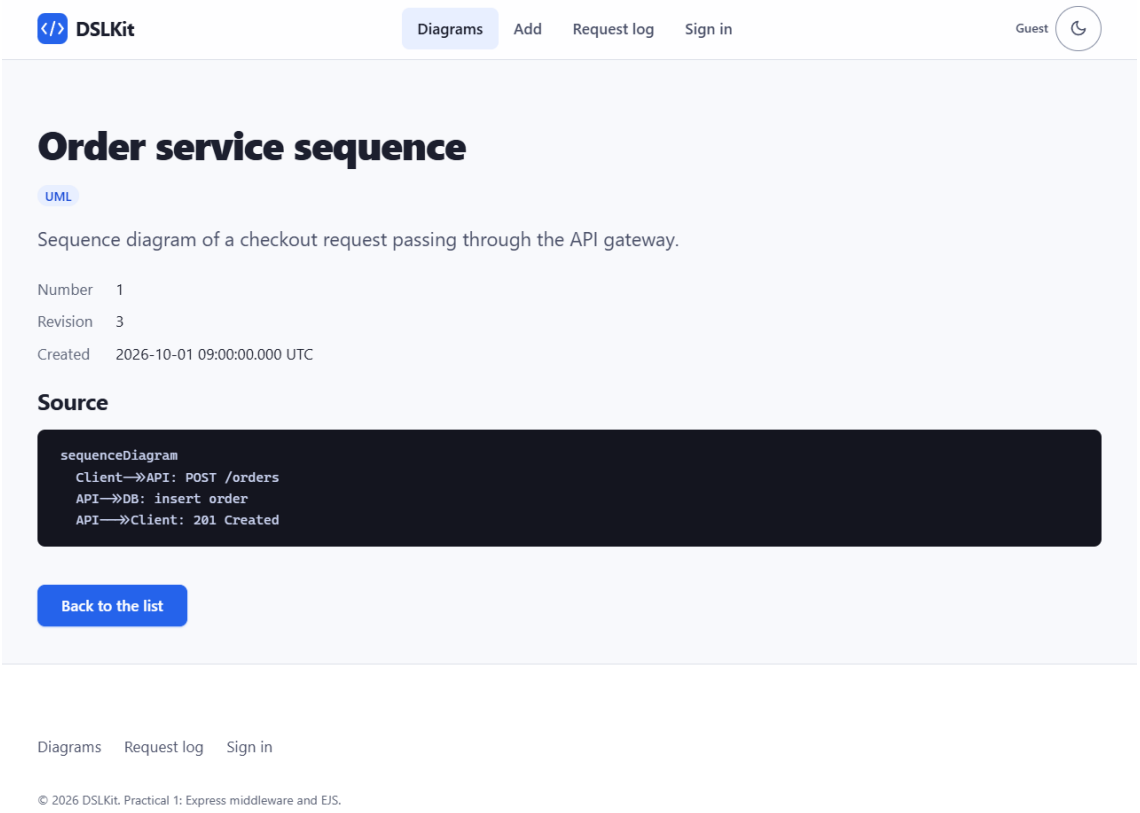


Рисунок 2 – Детальная страница диаграммы *Order service sequence*

Для проверки полей в форму добавления введены название из одних пробелов и ревизия минус 5. Результат показан на рисунке 3. Сервер вернул форму с кодом 400 и сообщениями под неверными полями, а введенное описание сохранилось.

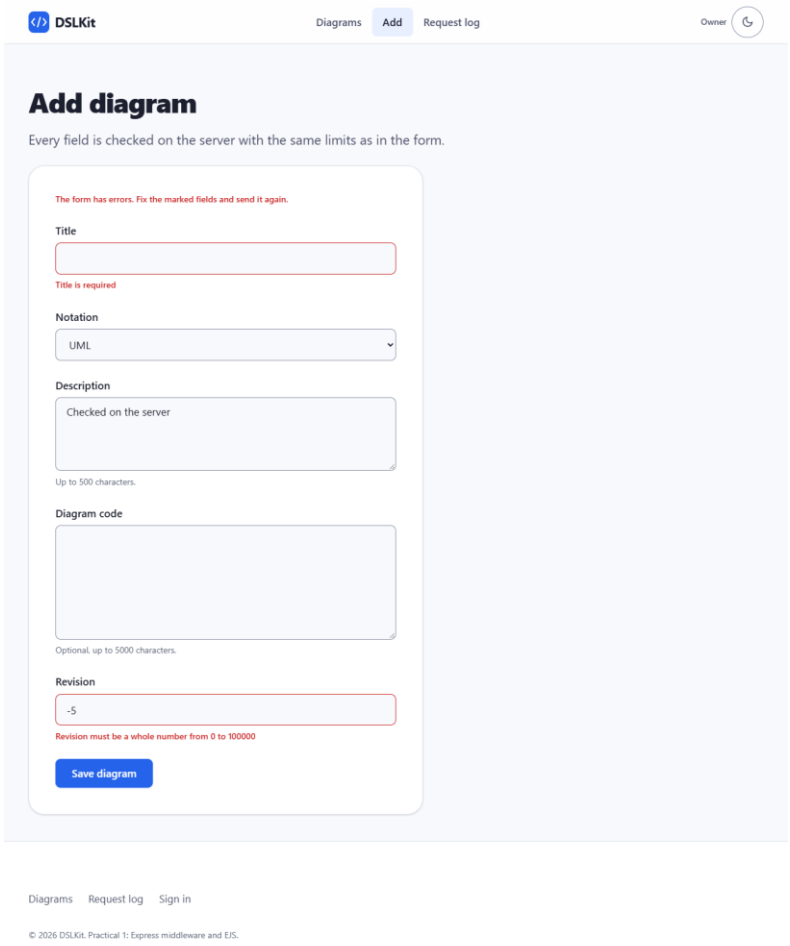


Рисунок 3 – Форма добавления с сообщениями о пустом названии и отрицательной ревизии

После исправления значений форму отправили повторно. Новая запись показана на рисунке 4. У карточки *Payment flow* свое время создания, поэтому записи различаются визуально.

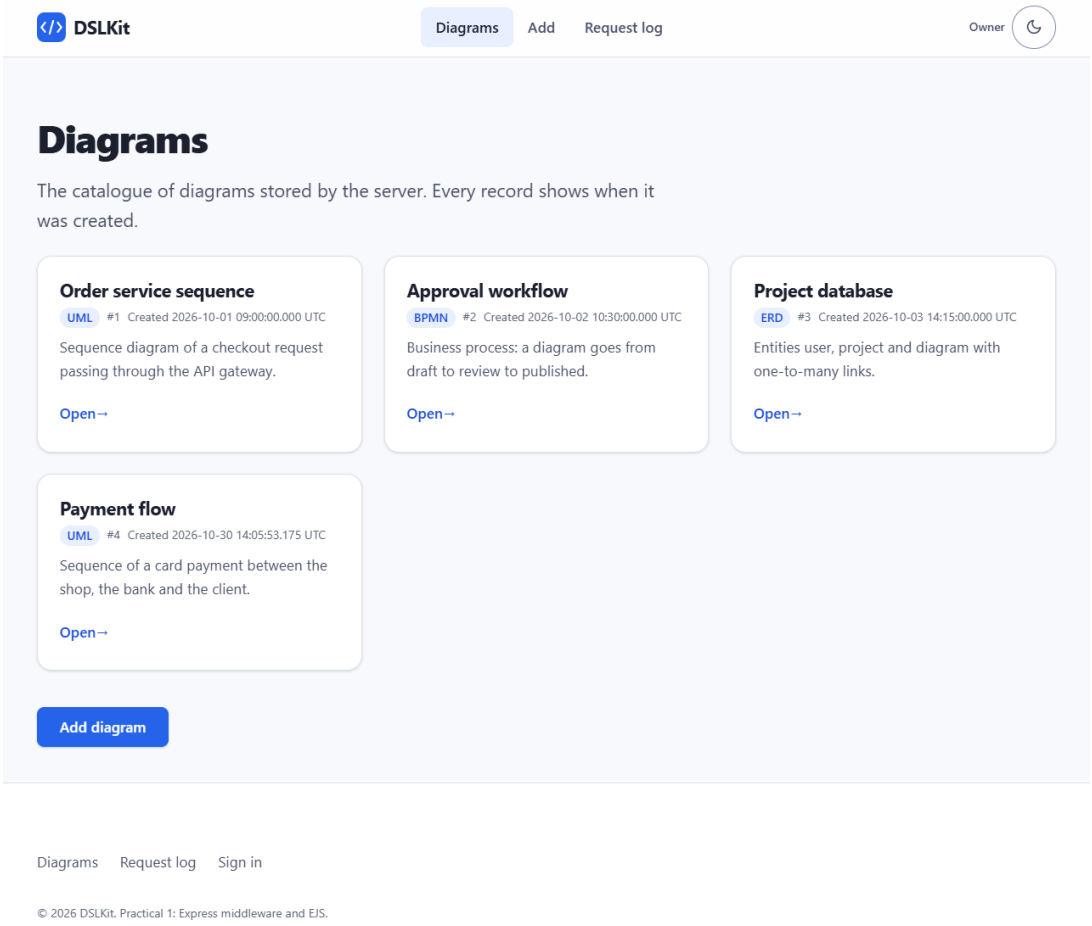


Рисунок 4 – Главная страница с новой диаграммой *Payment flow* после отправки формы

Форма изменения первой записи показана на рисунке 5. Поля заполнены текущими значениями и проверяются по тем же правилам, что при добавлении. Кнопка доступна владельцу, поэтому адрес содержит параметр *?auth=1*.

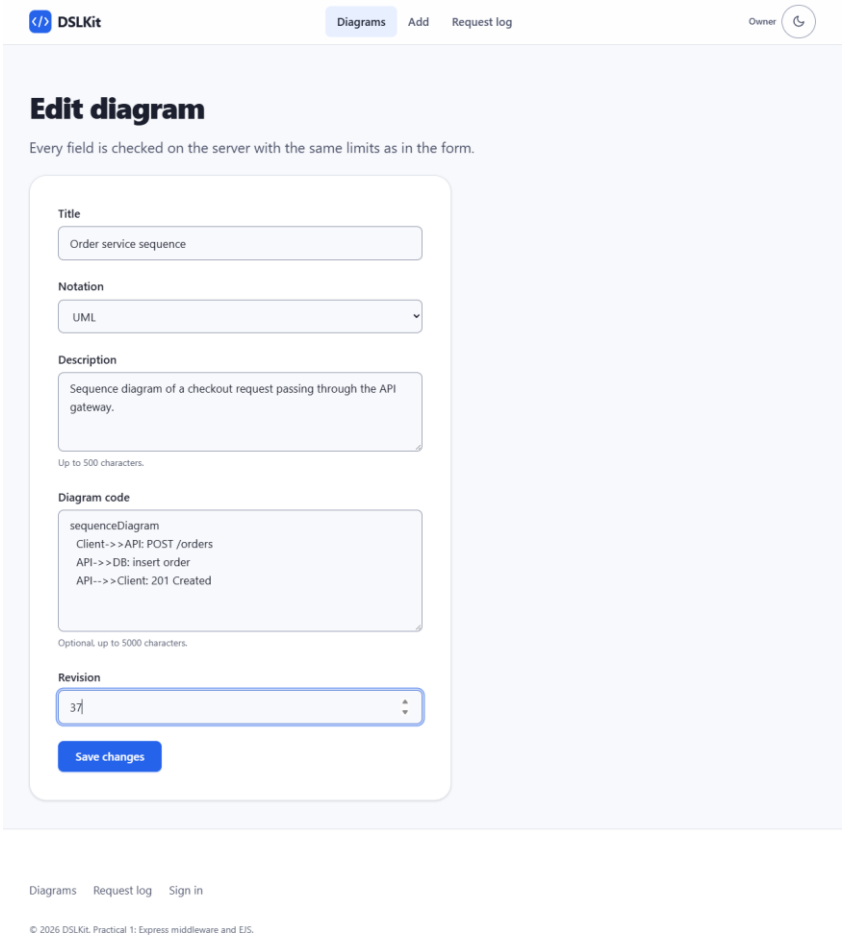


Рисунок 5 – Форма изменения первой диаграммы с текущими значениями полей

Результат сохранения показан на рисунке 6. Ревизия изменилась с 3 на 37, номер и время создания остались прежними, а сервер выполнил перенаправление с кодом 302.

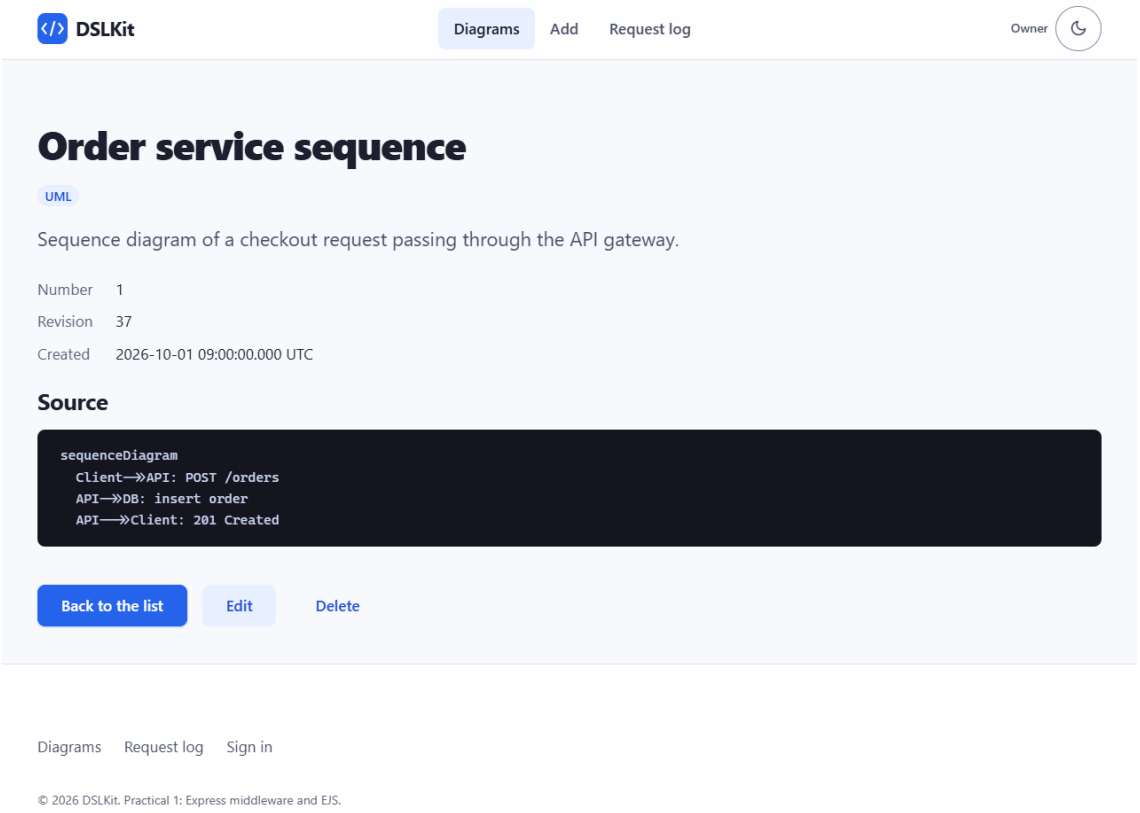


Рисунок 6 – Детальная страница первой диаграммы с ревизией 37 после изменения

Журнал запросов из таблицы *request_log* показан на рисунке 7. В строках указаны время, метод, адрес, код ответа и длительность, а значение параметра авторизации заменено звездочками.

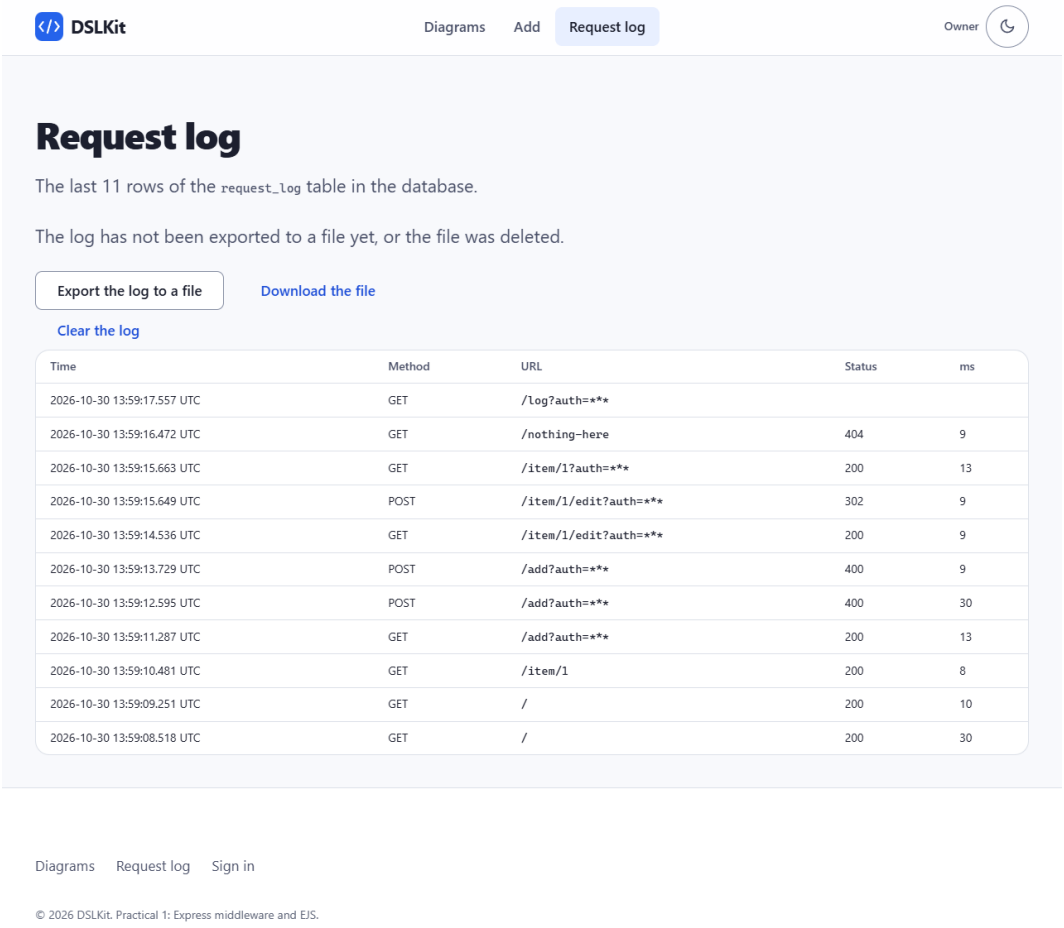


Рисунок 7 – Страница журнала с последними строками таблицы *request_log*

Страница ошибки для несуществующего адреса */nothing-here* показана на рисунке 8. В общем оформлении сайта она содержит три блока: что произошло, почему и как исправить.

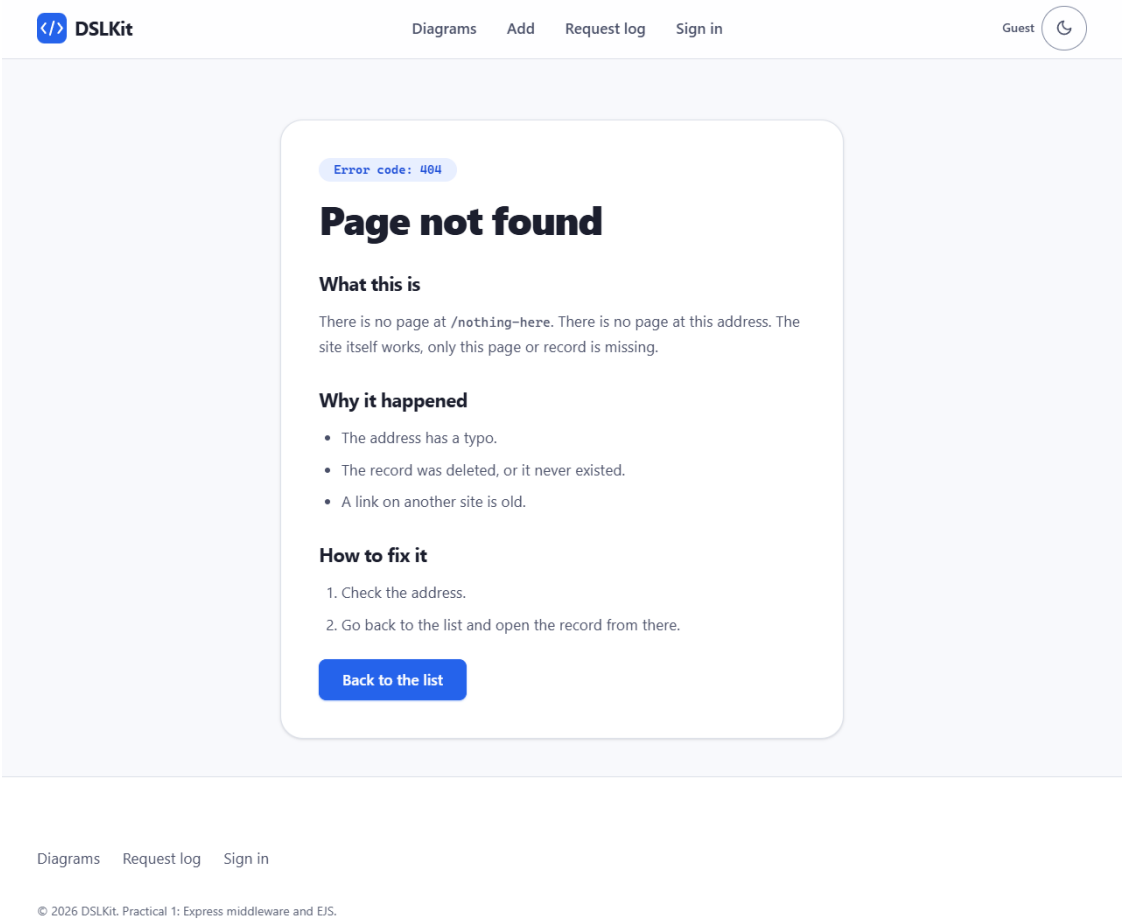


Рисунок 8 – Страница ошибки 404 для адреса */nothing-here*

Страница 500 показана на рисунке 9. Сервер запущен с путем *DB_FILE=package.json/blocked/log.db*, который проходит через обычный файл, поэтому база не открывается. Вместо системного сообщения *SQLite* пользователь видит причины и способы исправления.

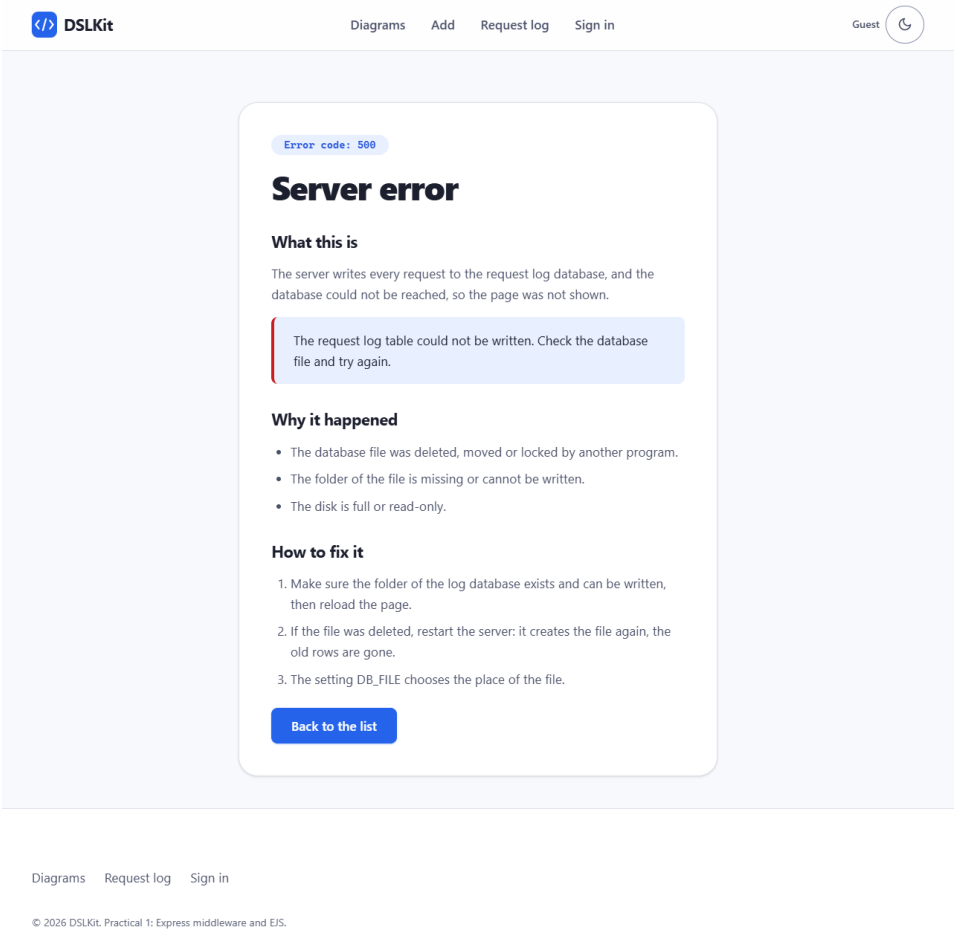


Рисунок 9 – Страница ошибки 500 при недоступной базе журнала

Страница 500 для случая, когда на месте файла базы создана папка, показана на рисунке 10. Вместо системного кода *EISDIR* она сообщает, что на месте файла журнала находится папка, и предлагает удалить или переименовать ее.

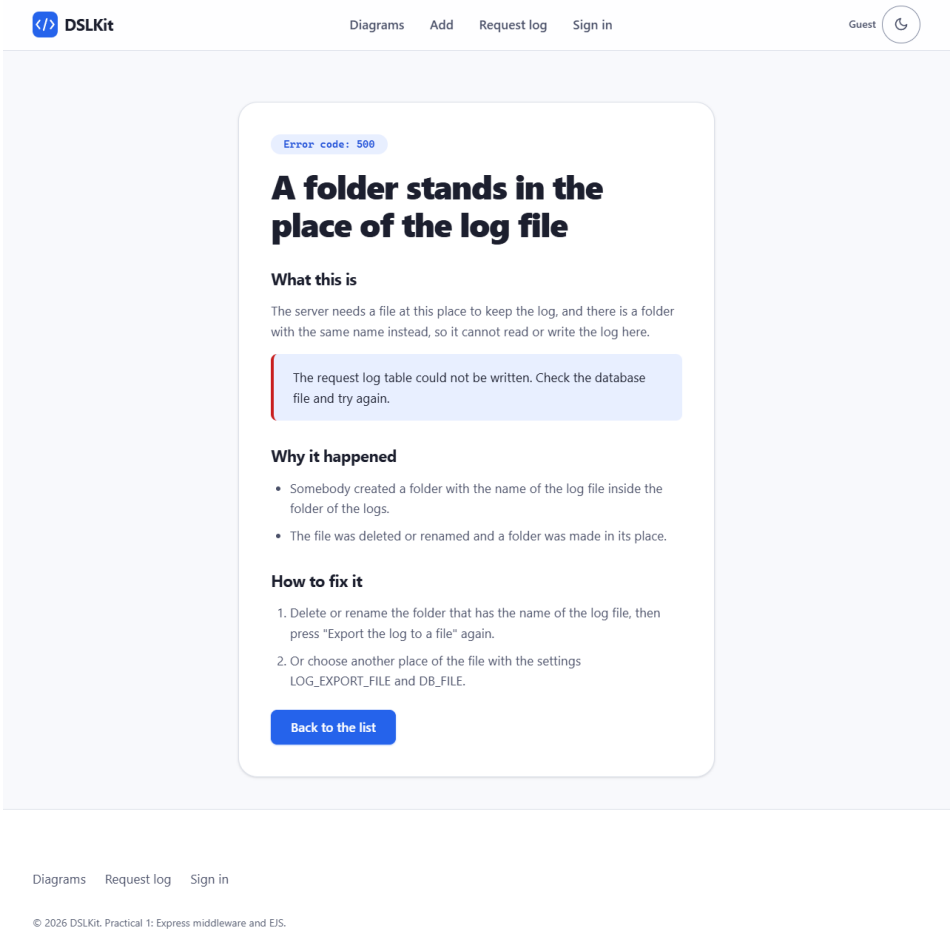


Рисунок 10 – Страница ошибки 500, когда на месте файла базы журнала создана папка

Вывод консоли сервера показан на рисунке 11. Каждую строку выводит *middleware* журнала: время, метод, адрес, код ответа и длительность.



Рисунок 11 – Вывод консоли сервера со строками журнала запросов

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Ниже даны ответы на три контрольных вопроса.

– Как в *Express* реализовать защиту от *CSRF* с помощью *middleware* (теоретически)?

Токен выдается при открытии формы и проверяется при отправке. *Middleware* создает случайное значение, кладет его в *cookie* с признаками *HttpOnly* и *SameSite=Strict*, а копию вставляет в скрытое поле формы. При запросе *POST* значения сравниваются, и при несовпадении сервер отвечает кодом 403. *Middleware* подключают после разбора тела формы и до маршрутов. В работе это функция *csrf* с полем *_csrf*.

– Как в *EJS* использовать функцию *helper* (например, для форматирования даты) в шаблоне?

Функцию помещают в *app.locals*, и она доступна во всех шаблонах, а для одного запроса ее кладут в *res.locals*. В работе *app.locals.formatTime* превращает строку *ISO* вида *2026-10-01T09:00:00.000Z* в 2026-10-01 09:00:00.000 *UTC*. В шаблоне ее вызывают записью *<%= formatTime(item.createdAt) %>*.

– Почему *middleware* для статики обычно подключается до всех маршрутов?

Запрос стилей или логотипа должен получить ответ сразу, без авторизации, журналирования и поиска среди маршрутов. Это ускоряет загрузку страницы и не добавляет в журнал строку на каждый файл. В работе *express.static* подключен после заголовков безопасности и до остальных *middleware*, поэтому в каталог *public* нельзя помещать закрытые данные.

ЗАКЛЮЧЕНИЕ

В ходе работы для проекта *DSLKit* создано приложение на *Express* 5 с шаблонами *EJS*. Страницы списка диаграмм, детальной записи и формы добавления получают данные из массива в памяти через *res.render()*, а общий каркас страниц собран из частичных шаблонов.

Реализованы собственные *middleware* журналирования, имитации авторизации и обработки ошибок. Журнал запросов по правилу четного варианта записывается в таблицу *SQLite*. Поломка файла или папки базы приводит к оформленной странице 500 с понятным пользователю пояснением вместо системного кода ошибки. Неизвестные адреса обрабатываются страницей 404.

Форма отвергает пустые и пробельные значения, слишком длинный текст и отрицательные числа по пределам из отдельного файла, а каждая запись получает уникальное время создания. Работоспособность подтверждена снимками страниц, сгенерированных сервером. Записи по условию задания хранятся в памяти и теряются при перезапуске сервера.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] *Using middleware* [Электронный ресурс] // *Express*. – Режим доступа : *https://expressjs.com/en/guide/using-middleware.html*.

[2] *Error handling* [Электронный ресурс] // *Express*. – Режим доступа : *https://expressjs.com/en/guide/error-handling.html*.

[3] *Embedded JavaScript templating* [Электронный ресурс] // *EJS*. – Режим доступа : *https://ejs.co/*.

[4] *SQLite* [Электронный ресурс] // *Node.js Documentation*. – Режим доступа : *https://nodejs.org/api/sqlite.html*.

[5] Практическое занятие 1. *Middleware* и шаблонизаторы : задание / Белорусский государственный университет информатики и радиоэлектроники. – Минск : БГУИР, 2026.

ПРИЛОЖЕНИЕ А
(обязательное)
Настройка шаблонизатора и порядок подключения *middleware*

```
import express from 'express';
import { fileURLToPath } from 'node:url';
import { createDiagramStore, validateDiagram, NOTATIONS } from
'./data/diagrams.js';
import { describeFile, exportLog, sendLogFile } from './data/logFile.js';
import { openRequestLog } from './data/requestLog.js';
import { requestLogger } from './middleware/requestLogger.js';
import { loadLimits } from './limits.js';
import { fakeAuth, requireAuth } from './middleware/fakeAuth.js';
import { notFound, serverError } from './middleware/errorHandlers.js';
import { csrf, rateLimit, securityHeaders } from './middleware/security.js';

const dir = (path) => fileURLToPath(new URL(path, import.meta.url));

const DEFAULT_DB_FILE = dir('../data/requests.db');
const DEFAULT_LOG_FILE = dir('../data/request-log.txt');

export const createApp = ({
  store = createDiagramStore(),
  limits = loadLimits(),
  log = openRequestLog(DEFAULT_DB_FILE),
  logFile = process.env.LOG_EXPORT_FILE || DEFAULT_LOG_FILE,
  write = console.log,
  writeError = console.error,
  now = Date.now,
} = {}) => {
  const app = express();
  app.set('view engine', 'ejs');
  app.set('views', dir('../views'));

  app.disable('x-powered-by');
  app.locals.limits = limits;
  // Records keep the exact ISO time for the machine; people read "2026-10-01
09:00:00.000 UTC".
  app.locals.formatTime = (iso) => String(iso).replace('T', ' ').replace('Z', ' UTC');

  // Order matters: headers first so every answer has them, then static files and the pure
checks,
  // then the rate limits (a flood is refused before it reaches the database), then the logger
(which can fail
  // and hand over to the error handler), the body parser and the CSRF check (the token is
a form field), then the routes.
  // The two error handlers are the very last, so every failure ends on a styled page.
```

```
  app.use(securityHeaders);
  // Liveness probe for Docker and monitoring: placed before the rate limit and the log, so
probes neither use up the quota nor fill the table.
  app.get('/health', (req, res) => res.json({ status: 'ok' }));
  app.use(express.static(dir('../public')));
  app.use((req, res, next) => {
    res.locals.path = req.path;
    next();
  });
  app.use(fakeAuth);
  app.use(rateLimit({ ...limits.rateLimit.all, now }));
  app.use(rateLimit({ ...limits.rateLimit.write, only: ['POST'], now }));
  app.use(requestLogger(log, write, limits.logUrlMax));
```

ПРИЛОЖЕНИЕ Б
(обязательное)
Шаблон главной страницы со списком диаграмм

```
<%- include('partials/header') %>
<section class="section" aria-labelledby="page-title">
  <h1 class="section__title" id="page-title">Diagrams</h1>
  <p class="section__lead">The catalogue of diagrams stored by the server. Every
record shows when it was created.</p>
  <% if (items.length === 0) { %>
    <p class="note">No diagrams yet.</p>
  <% } else { %>
    <ul class="cards">
      <% items.forEach((item) => { %>
        <li class="cards__item">
          <article class="card">
            <h2 class="card__title"><%= item.title %></h2>
            <p class="card__meta">
              <span class="badge"><%= item.notation %></span>
              <span>#<%= item.id %></span>
              <span>Created <time datetime="<%= item.createdAt %>"><%=
formatTime(item.createdAt) %></time></span>
            </p>
            <p class="card__text"><%= item.description %></p>
            <a class="card__link" href="/item/<%= item.id %><%= authQuery
%>">Open<span class="visually-hidden"> <%= item.title %></span></a>
          </article>
        </li>
      <% }) %>
    </ul>
  <% } %>
  <div class="page-actions">
    <a class="button button--primary" href="/add<%= authQuery %>">Add
diagram</a>
  </div>
</section>
<%- include('partials/footer') %>
```

ПРИЛОЖЕНИЕ В
(обязательное)
Middleware записи запросов в журнал

```
import { UserError } from '../errors.js';
import { safeLogText } from './security.js';

// Writes the request into the request_log table BEFORE the route runs, so a database failure
// reaches the error handler and the user sees the styled 500 page. Status and duration are added
// to the same row when the response finishes.
// The sign-in flag of the imitation must not be copied into the log, so a reader of the log cannot reuse it.
export const hideAuth = (url) => String(url).replace(/([?&]auth=)[^&#]*/gi, '$1***');

export const requestLogger =
  (log, write = console.log, urlMax = 500) =>
  (req, res, next) => {
    const startedAt = Date.now();
    // The address is user input: line breaks and control characters are escaped and the length is cut (log injection).
    const url = safeLogText(hideAuth(req.originalUrl), urlMax);
    let id;
    try {
      id = log.start({ time: new Date(startedAt).toISOString(), method: req.method, url });
    } catch (cause) {
      // The database layer already names the problem (UserError); anything else is wrapped in a plain error.
      const error = cause instanceof UserError ? cause : new Error('Request log write failed', { cause });
      error.publicMessage = 'The request log table could not be written. Check the database file and try again.';
      return next(error);
    }
    res.on('finish', () => {
      const ms = Date.now() - startedAt;
      try {
        log.finish(id, res.statusCode, ms);
      } catch (error) {
        write(`[log] could not finish row ${id}: ${error.message}`);
      }
      write(`[${new Date(startedAt).toISOString()}] ${req.method} ${url} ${res.statusCode} ${ms}ms`);
    });
    return next();
  };
```

ПРИЛОЖЕНИЕ Г
(обязательное)
Создание таблицы журнала и запросы к базе *SQLite*

```
import { DatabaseSync } from 'node:sqlite';
import { existsSync, mkdirSync, statSync } from 'node:fs';
import { dirname } from 'node:path';
import { UserError, fileFailure } from '../errors.js';

const CREATE_TABLE = `
  CREATE TABLE IF NOT EXISTS request_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    created_at TEXT NOT NULL,
    method TEXT NOT NULL,
    url TEXT NOT NULL,
    status INTEGER,
    duration_ms INTEGER
  )`;

// The request log is a table in a SQLite database. The connection opens lazily on first use,
// so a database that cannot be opened fails the request (and shows the 500 page), not the whole server.
export const openRequestLog = (file = ':memory:') => {
  let db = null;
  let statements = null;
  const connect = () => {
    if (statements) return statements;
    // The folder is made only when it is missing: mkdir on the root of a drive (E:/) fails on Windows even with recursive.
    if (file !== ':memory:' && !existsSync(dirname(file))) mkdirSync(dirname(file), { recursive: true });
    if (file !== ':memory:' && existsSync(file) && statSync(file).isDirectory()) throw Object.assign(new Error('a folder stands in the place of the file'), { code: 'EISDIR' });
    const opened = new DatabaseSync(file);
    try {
      opened.exec(CREATE_TABLE);
      statements = {
        insert: opened.prepare('INSERT INTO request_log (created_at, method, url) VALUES (?, ?, ?)'),
        finish: opened.prepare('UPDATE request_log SET status = ?, duration_ms = ? WHERE id = ?'),
        clear: opened.prepare('DELETE FROM request_log'),
        recent: opened.prepare('SELECT id, created_at AS time, method, url, status, duration_ms AS ms FROM request_log ORDER BY id DESC LIMIT ?'),
      };
    } catch (error) {
```

```
      opened.close();
      throw error;
    }
    db = opened;
    return statements;
  };
  // Any failure of the database (a deleted or locked file, a missing folder, a full disk) becomes the error page about the log database.
  const guard =
    (action) =>
    (...args) => {
      try {
        return action(...args);
      } catch (error) {
        throw error instanceof UserError ? error : new UserError(fileFailure(error, 'log-database-unavailable'), error);
      }
    };
  return {
    start: guard(({ time, method, url }) => Number(connect().insert.run(time, method, url).lastInsertRowid)),
    finish: guard((id, status, ms) => connect().finish.run(status, ms, id)),
    // Removes every row and returns how many were removed.
    clear: guard(() => Number(connect().clear.run().changes)),
    recent: guard((limit) =>
      connect()
        .recent.all(limit)
        .map((row) => ({ ...row })),
    ),
    close: () => {
      // A second close must not throw: the connection is closed once and then forgotten.
      if (db) db.close();
      db = null;
    },
  };
};
```

ПРИЛОЖЕНИЕ Д
(обязательное)
Middleware имитации авторизации

```
// Simulated authorization: ?auth=1 means "signed in as the owner", anything else is a guest.
export const fakeAuth = (req, res, next) => {
  const signedIn = req.query.auth === '1';
  req.user = signedIn ? { name: 'Owner', guest: false } : { name: 'Guest', guest: true };
  res.locals.user = req.user;
  res.locals.authQuery = signedIn ? '?auth=1' : '';
  next();
};

// Guards a single route: guests are redirected to /login.
export const requireAuth = (req, res, next) => (req.user.guest ? res.redirect('/login') : next());
```

ПРИЛОЖЕНИЕ Е
(обязательное)
Разбор тела формы и обработчики маршрутов приложения

```
  app.use(express.urlencoded({ extended: false, limit: limits.bodyLimit, parameterLimit: limits.parameterLimit }));
  app.use(csrf);

  app.get('/', (req, res) => res.render('index', { title: 'Diagrams', items: store.all() }));

  // Only plain digits are a number: Number() would also accept "0x1" or "1e0".
  const findItem = (req) => (/^\d{1,9}$/.test(req.params.id) ? store.find(Number(req.params.id)) : undefined);

  // Adding and editing share one template, only the texts and the address differ.
  const formView = { notations: NOTATIONS, heading: 'Add diagram', action: '/add', submit: 'Save diagram' };
  const editView = (item) => ({ ...formView, heading: 'Edit diagram', action: `/item/${item.id}/edit`, submit: 'Save changes' });

  // Repeated keys (arrays) must not crash: only plain strings count.
  const readForm = (body) => {
    const text = (value) => (typeof value === 'string' ? value.trim() : '');
    return {
      title: text(body.title),
      notation: text(body.notation),
      description: text(body.description),
      code: text(body.code),
      revision: text(body.revision),
    };
  };

  app.get('/item/:id', (req, res, next) => {
    const item = findItem(req);
    return item ? res.render('item', { title: item.title, item }) : next();
  });

  // The address of one record answers only to GET: another method is a mistake of the client, not a missing page.
  app.all('/item/:id', (req, res, next) => {
    if (req.method === 'GET' || req.method === 'HEAD') return next();
    res.set('Allow', 'GET, HEAD');
    return next(
      Object.assign(new Error('Method not allowed'), { status: 405, publicMessage: 'This address only answers GET. To change the record use its edit form.' }),
    );
  });
```

```
  app.get('/add', requireAuth, (req, res) => res.render('add', { ...formView, title: 'Add
diagram', errors: {}, values: { revision: '1' } }));

  app.post('/add', requireAuth, (req, res) => {
    const values = readForm(req.body); // the CSRF check has already proved that the
form body exists
    const errors = validateDiagram(values, limits);
    if (Object.keys(errors).length) {
      return res.status(400).render('add', { ...formView, title: 'Add diagram', errors, values
});
    }
    store.add({ ...values, revision: Number(values.revision) });
    return res.redirect(`/${res.locals.authQuery}`);
  });

  // Edit and delete change data, so both need the simulated sign-in and the CSRF token
(the delete button is a form, not a link).
  app.get('/item/:id/edit', requireAuth, (req, res, next) => {
    const item = findItem(req);
    if (!item) return next();
    return res.render('add', { ...editView(item), title: 'Edit diagram', errors: {}, values: {
...item, revision: String(item.revision) } });
  });

  app.post('/item/:id/edit', requireAuth, (req, res, next) => {
    const item = findItem(req);
    if (!item) return next();
    const values = readForm(req.body);
    const errors = validateDiagram(values, limits);
    if (Object.keys(errors).length) {
      return res.status(400).render('add', { ...editView(item), title: 'Edit diagram', errors,
values });
    }
    store.update(item.id, { ...values, revision: Number(values.revision) });
    return res.redirect(`/item/${item.id}${res.locals.authQuery}`);
  });

  app.post('/item/:id/delete', requireAuth, (req, res, next) => {
    const item = findItem(req);
    if (!item) return next();
    store.remove(item.id);
    return res.redirect(`/${res.locals.authQuery}`);
  });

  app.get('/login', (req, res) => res.render('login', { title: 'Sign in' }));
  app.get('/log', requireAuth, async (req, res, next) => {
    try {
```

```
    res.render('log', { title: 'Request log', rows: log.recent(limits.logRows), file: await
describeFile(logFile), exported: req.query.exported === '1' });
   } catch (error) {
    next(error);
   }
  });

  // The log can be written to a text file and downloaded; both steps use streams and every
failure has its own page (see logFile.js and errors.js).
  app.post('/log/export', requireAuth, async (req, res, next) => {
   try {
    await exportLog(log, logFile);
    res.redirect(`/log?exported=1${res.locals.authQuery.replace('?', '&')}`);
   } catch (error) {
    next(error);
   }
  });

  app.get('/log/download', requireAuth, async (req, res, next) => {
   try {
    await sendLogFile(res, logFile, writeError);
   } catch (error) {
    next(error);
   }
  });

  // Clearing the log changes data, so it needs the simulated sign-in and the CSRF token,
and it is a form, not a link.
  app.post('/log/clear', requireAuth, (req, res) => {
   log.clear();
   return res.redirect(`/log${res.locals.authQuery}`);
  });

  app.use(notFound);
  app.use(serverError(writeError));
  return app;
};
```

ПРИЛОЖЕНИЕ Ж
(обязательное)
Шаблон детальной страницы диаграммы

```
<%- include('partials/header') %>
<section class="section" aria-labelledby="page-title">
  <h1 class="section__title" id="page-title"><%= item.title %></h1>
  <p><span class="badge"><%= item.notation %></span></p>
  <p class="section__lead"><%= item.description %></p>
  <dl class="facts">
    <dt>Number</dt>
    <dd><%= item.id %></dd>
    <dt>Revision</dt>
    <dd><%= item.revision %></dd>
    <dt>Created</dt>
    <dd><time datetime="<%= item.createdAt %>"><%= formatTime(item.createdAt)
%></time></dd>
  </dl>
  <% if (item.code) { %>
    <h2>Source</h2>
    <pre class="code"><%= item.code %></pre>
  <% } %>
  <div class="page-actions">
    <a class="button button--primary" href="/<%= authQuery %>">Back to the
list</a>
    <% if (!user.guest) { %>
      <a class="button button--tonal" href="/item/<%= item.id %>/edit<%= authQuery
%>">Edit<span class="visually-hidden"> <%= item.title %></span></a>
      <form class="inline-form" method="post" action="/item/<%= item.id
%>/delete<%= authQuery %>">
        <input type="hidden" name="_csrf" value="<%= csrfToken %>" />
        <button class="button button--quiet" type="submit">Delete<span class="visually-
hidden"> <%= item.title %></span></button>
      </form>
    <% } %>
  </div>
</section>
<%- include('partials/footer') %>
```

ПРИЛОЖЕНИЕ И
(обязательное)
Обработчики ошибок 404 и 500

```
import { CATALOG, entryFor } from '../errors.js';
import { safeLogText } from './security.js';

export const notFound = (req, res) => {
  res.status(404).render('404', { title: 'Not found', url: req.originalUrl, ...CATALOG[404]
});
};

// Last resort when even the 500 template fails: still a styled page, never a bare text
screen.
const fallbackPage = (status) =>
  `<!DOCTYPE html><html lang="en"><head><meta charset="utf-
8"><title>${status} – DSLKit</title><link rel="stylesheet" href="/dslkit.css"><link
rel="stylesheet" href="/pz.css"></head>` +
  `<body class="page"><main class="main"><section class="error"><p
class="error__code">Error code: ${status}</p><h1 class="error__title">${status} –
server error</h1><p class="error__text">The error page could not be drawn. The
error was logged.</p><a class="button button--primary" href="/">Back to the
list</a></section></main></body></html>`;

// Four parameters mark this as the error-handling middleware. It is mounted after every
route and
// middleware, so any failure ends on the 500 page (or the 4xx variant for client errors).
export const serverError =
  (write = console.error) =>
  (err, req, res, _next) => {
    write(`[error] ${safeLogText(req.method)} ${safeLogText(req.originalUrl)}`, err);
    if (res.headersSent) return res.end();
    if (err.retryAfter) res.set('Retry-After', String(err.retryAfter));
    // Errors from body-parser and others carry their own 4xx/5xx status.
    const status = err.status >= 400 && err.status < 600 ? err.status : 500;
    // The page tells what the error is, why it happened and how to solve it (see errors.js).
    const { heading, what, why, fix } = entryFor(err, status);
    const locals = {
      title: heading,
      heading,
      status,
      message: err.publicMessage ?? '',
      what,
      why,
      fix,
      // The failure may happen before fakeAuth ran, so the page needs its own defaults.
      user: res.locals.user ?? { name: 'Guest', guest: true },
```

```
    authQuery: res.locals.authQuery ?? '',
    path: '',
  };
  return res.status(status).render('500', locals, (renderError, html) => {
    if (renderError) {
      write('[error] the 500 page failed to render', renderError);
      return res.type('html').send(fallbackPage(status));
    }
    return res.send(html);
  });
};
```

ПРИЛОЖЕНИЕ К
(обязательное)
Хранилище записей и проверка полей формы

```
export const NOTATIONS = ['UML', 'BPMN', 'ERD', 'DFD', 'IDEF0', 'Petri'];

const seed = () => [
  {
    id: 1,
    title: 'Order service sequence',
    notation: 'UML',
    description: 'Sequence diagram of a checkout request passing through the API
gateway.',
    code: 'sequenceDiagram\n  Client->>API: POST /orders\n  API->>DB: insert order\n
API-->>Client: 201 Created',
    revision: 3,
    createdAt: '2026-10-01T09:00:00.000Z',
  },
  {
    id: 2,
    title: 'Approval workflow',
    notation: 'BPMN',
    description: 'Business process: a diagram goes from draft to review to published.',
    code: 'start -> draft -> review -> published',
    revision: 1,
    createdAt: '2026-10-02T10:30:00.000Z',
  },
  {
    id: 3,
    title: 'Project database',
    notation: 'ERD',
    description: 'Entities user, project and diagram with one-to-many links.',
    code: 'user 1--* project\nproject 1--* diagram',
    revision: 0,
    createdAt: '2026-10-03T14:15:00.000Z',
  },
];

// Data lives in memory only, as the practical requires: no database for the diagrams.
// Every record gets its own creation time: it is always later than the previous one, even
within one millisecond.
export const createDiagramStore = (now = Date.now) => {
  const items = seed();
  let nextId = items.length + 1;
  let last = Math.max(...items.map((item) => Date.parse(item.createdAt)));
  return {
    all: () => [...items],
```

```
    find: (id) => items.find((item) => item.id === id),
    add: ({ title, notation, description, code, revision }) => {
      last = Math.max(now(), last + 1);
      const item = { id: nextId, title, notation, description, code, revision, createdAt: new
Date(last).toISOString() };
      nextId += 1;
      items.push(item);
      return item;
    },
    // Edit keeps the id and the creation time, only the form fields change.
    update: (id, { title, notation, description, code, revision }) => {
      const item = items.find((one) => one.id === id);
      if (!item) return undefined;
      Object.assign(item, { title, notation, description, code, revision });
      return item;
    },
    remove: (id) => {
      const index = items.findIndex((one) => one.id === id);
      if (index < 0) return false;
      items.splice(index, 1);
      return true;
    },
  };
};

const CONTROL = /[\u0000-\u001f• \u202a-\u202e\u2066-\u2069]/;

const required = (label, value, max) => {
  if (value === '') return `${label} is required`;
  if (value.length > max) return `${label} must be at most ${max} characters`;
  return null;
};

// Values arrive already trimmed, so a whitespace-only value is an empty one.
// Returns an object of field errors, empty when the input is valid. Limits come from
limits.json.
export const validateDiagram = (input, limits) => {
  const errors = {};
  const set = (field, message) => message && (errors[field] = message);
  set('title', required('Title', input.title, limits.title.maxLength));
  // Control characters (NUL, escape) and text-direction overrides are not part of a title:
they break lists and can flip the text around it.
  if (!errors.title && CONTROL.test(input.title)) errors.title = 'Title must not contain
control characters';
  set('description', required('Description', input.description,
limits.description.maxLength));
  if (input.code.length > limits.code.maxLength) errors.code = `Code must be at most
${limits.code.maxLength} characters`;
```

```
  if (!NOTATIONS.includes(input.notation)) errors.notation = 'Choose a notation from
the list';
  const { min, max } = limits.revision;
  if (input.revision === '') errors.revision = 'Revision is required';
  else if (!/^\d+$/.test(input.revision) || Number(input.revision) < min ||
Number(input.revision) > max) {
    errors.revision = `Revision must be a whole number from ${min} to ${max}`;
  }
  return errors;
};
```

ПРИЛОЖЕНИЕ Л
(обязательное)
Файл пределов для полей формы и журнала

```
{
  "title": { "maxLength": 80 },
  "description": { "maxLength": 500 },
  "code": { "maxLength": 5000 },
  "revision": { "min": 0, "max": 100000 },
  "bodyLimit": "20kb",
  "parameterLimit": 20,
  "logRows": 100,
  "logUrlMax": 500,
  "rateLimit": {
    "all": { "windowMs": 60000, "max": 300 },
    "write": { "windowMs": 60000, "max": 20 }
  }
}
```